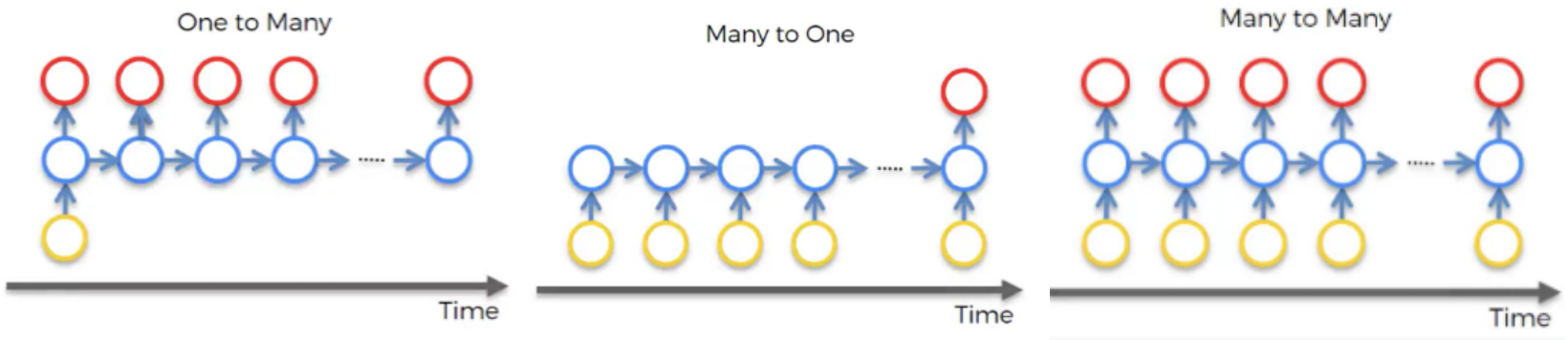
For this lab:

- You will need to understand the material in the notes on **recurrent neural networks**.
- A code and data zip file that is essential for doing this lab can be found **here**. Download this to your computer. Alternatively, you can use a **colab notebook**.

RNNs

1) Examples of RNN Application

In this section, we'll consider examples of RNN applications with respect to the forms of inputs and outputs.



Choose an RNN structure from among one-to-one, many-to-one, and many-to-many, to address each of the problems below. For each, describe how you would structure the input to the RNN, and what kind of output unit(s) you would use.

1A) Given a News article on Apple's new iphone, detect whether the text's sentiment is positive or negative.

1B) Assign a part-of-speech tag to each word in an English sentence "I am a boy"; "subject, verb, article, noun"

1C) Given a picture of a cat sitting on a rock, generate a caption "A cat is sitting on a rock".

2) Accumulator

In the exercises, we have seen **recurrent** neural network (RNN) models. Recall the basic RNN equations:

$$s_t = f_1(W^{ss} s_{t-1} + W^{sx} x_t + W_0^{ss})$$
$$p_t = f_2(W^{o} s_t + W_0^{o})$$

where x_t are the inputs and p_t are the "predicted" outputs and f_1 and f_2 are activation functions, such as tanh and softmax. In this model we have also included explicitly the offset parameter vectors W_0^{ss} $(m \times 1)$ and W_0^{o} $(n \times 1)$ where m is the dimension of the state space and n the dimension of the output space. Typically the starting state s_0 is set to an all-zero vector.

In HW 9 we saw the `Accumulator` state machine, which adds up (accumulates) its input and outputs the sum.

```
class Accumulator(SM):
    start_state = 0
    def transition_fn(self, s, i):
        return s + i
    def output_fn(self, s):
        return s
```

We can use an RNN to try to learn a model of the mapping from input sequences to output sequences defined by this machine.

2A) What are the values of parameters for an RNN that performs the same operation as the Accumulator machine? Also, specify the shape of the input, output and state vectors, the activation functions and the weight and bias matrices.

2B) In file `code_for_lab11.py`, there is a function called `test_linear_accumulator`; this function trains an RNN with input and output sequences of the kind produced by Accumulator. (As its name suggests, the outputs of the trained RNN and Accumulator should be comparable.)

Run the function a few times, making sure that the training error is low and look at the final weights. Relate to the weights you chose above. Explain.

☐ Check this box and submit when you have finished all parts of this question.

Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

3) Generating sequences

We are interested in building an RNN model that can predict the next element in a sequence (this is sometimes referred to as a "language" model). The particular form of the model that we will look at is:

$$
\begin{aligned}
x_t &= \phi(y_{t-1}) \\
s_t &= \tanh(W^{ss} s_{t-1} + W^{sx} x_t + W_0^{ss}) \\
p_t &= \text{softmax}(W^{o} s_t + W_0^{o})
\end{aligned}
$$

where

- we are working with a vocabulary of size V consisting of distinct atomic symbols (here characters),
- y_t corresponds to the desired output at time t in the sequence and is one of the V atomic symbols in our vocabulary,
- $\phi(y_t)$ is the one-hot vector of dimension V corresponding to symbol y_t,
- y_{t-1} is the desired output at time $t-1$ in the sequence, and $\phi(y_{t-1})$, the one-hot vector representation of y_{t-1}, is used as x_t, the input at time t,
- W_0^{ss} is $m \times 1$ where m is the dimension of the state space,
- W_0^{o} is $V \times 1$,
- p_t is a vector of probabilities over the V possible symbols,
- typically the starting state s_0 is set to an all-zero vector,
- '.' and '\n' are delimeters that denote the end or start of a string.
- y_0 is set to a start symbol ('.' or '\n'), which are also included in the vocabulary.

In order for our model to generate finite length sentences, we also include an 'end' symbol ('.' or '\n') which, when generated from the model (sampled from p_t), simply halts the process.

We'll try to understand, learn, and improve these types of models.

In file `code_for_lab11.py` you will find definitions of a procedure for training and using models of sequential data.

```
generate_seq(data, num_hidden=20, step_size=0.05, num_steps=10000, split=0,
interactive=False)
```

- `data` is a list of character strings. These will be converted into a training set of sequence pairs for a language model as described in the notes.
- `num_hidden` indicates the number of units in the hidden layer (the dimension of the states),
- `num_steps` indicates steps of (stochastic) gradient descent,
- `step_size` the magnitude of the gradient descent steps,
- `split` is a percentage of the data to be held out, for estimating loss,
- `interactive` indicates, when False, to generate 100 random sequences from learned model, when True, it asks for a partial sequence and then completes it in the most likely way given the learned model.

Training is as follows:

- For each sequence in the input data, it feeds in character $t-1$ from the training data and predicts character t.

Generation is as follows:

- Starting with the *start* symbol ('.'), it predicts a next character based on the softmax distribution in the trained model, then it feeds that character into the model and repeats until an *end* symbol ('.') is generated.

We will first see how well these models can learn to produce a single string. As a measure of difficulty of learning, let's

consider two factors:

- num_hidden: The dimension of the hidden state,
- num_steps: The number of steps. For this first example, try num_steps in [1000, 5000, 10000, 15000, 20000].

Let's consider difficulty to be ordered lexicographically by the combination of (num_hidden, num_steps) required to accurately reproduce the word 100 times, for a given state dimension.

Note that the initial weights are chosen randomly, so results will vary for each run. You can set the random seed if you can't get consistent results.

We'll consider these strings:

- "aaaaaaaaaa"
- "aabaaabbaaaababaabaa"
- "abcdefghijklmnopqrstuvwxyz"
- "abcabcabcabcabc"

3A) Which two do you think will be most difficult to learn? Why? ("Difficult" would consist of needing larger hidden dimension or number of training steps)

3B) What do you think would be the smallest possible value of num_hidden (the hidden state dimension) to be able to successfully learn the mapping needed for "aaaaaaaaaa"? Would we expect it to be easier or harder to learn with that value for num_hidden or for a larger value?

3C) Try learning each of these strings, using the `test_word` function for different values of num_hidden and

num_steps. Find the difficulty of each string (the minimum size of hidden layer and number of steps required to consistently reproduce the string, where we prioritize hidden layer size). Comment on your results.

☐ Check this box and submit when you have finished problems 3A through 3C.

Submit | View Answer | Ask for Help

You have infinitely many submissions remaining.

Try the following so you can see what's possible with these relatively simple models:

3D) The `test_class_names` function uses the file MIT_classes.txt (which is a collection of names of classes taught at MIT) for training and generates new names. Experiment with different values of num_steps; more steps gives better results. Try it in interactive mode, by setting `interactive=True` but still set `interactive_top5=False`; it's more fun. Note the difference between starting with a capital letter versus lower case letter.

For each of the following, again experiment with num_steps for training, and play with non-interactive and/or interactive mode outputs to see the results.

3E) The `test_food` function uses the file food.txt of recipe names.

3F) The `test_company_names` function uses the file companies.txt of company names.

3G) Now run one of `test_class_names, test_food, test_company_names` with `interactive_top5=True`; play with them for a while. What is the mechanism by which these RNNs generate their

output? More specifically, what character does the trained RNN seem to output at each location? Also, what would be a reasonable criterion for the trained RNN to (or when does it seem to) stop generating more characters?

3H) Observe and compare the behavior of `test_class_names` and `test_company_names`. Why do you think there is such a difference? (You may need to consider and look at their respective training files as well.)

3I) What do the RNNs constructed in `test_class_names` and `test_company_names` correspond to among the "one-to-many", "many-to-one", and "many-to-many" from section 1?

Note: If your installation of Python has trouble finding the data files, try setting the `dirname` in the `code_for_lab11.py` file to the pathname for the folder where the data can be found.